WEEK 2: SQL LOGIC + MODELLING (POSTGRESQL)

PostgreSQL After SQLite

- SQLite is embedded and file-based.
- PostgreSQL is server-based and supports concurrent users.
- PostgreSQL has stronger support for constraints, joins, and advanced SQL.
- Docker gives the class a consistent setup.
- The VS Code extension lets you run queries inside the editor.

Subqueries

- A subquery is a query inside another query.
- In the PostgreSQL tutorial, it solves a query you cannot write directly.

```
SELECT city
FROM weather
WHERE temp_lo = (
  SELECT max(temp_lo) FROM weather
);
```

CTEs

- A CTE is a named intermediate result set.

```
WITH regional_sales AS (
  SELECT region, SUM(amount) AS total_sales FROM orders GROUP BY region
), top_regions AS (
  SELECT region FROM regional_sales
  WHERE total_sales > (SELECT SUM(total_sales) / 10 FROM regional_sales)
)
SELECT region FROM top_regions;
```

JOIN Types

- `INNER JOIN` : matching rows only.
- `LEFT JOIN` : all left rows, plus matches from the right.
- `RIGHT JOIN` : all right rows, plus matches from the left.
- `FULL OUTER JOIN` : rows from both sides.
- Prefer explicit `JOIN ... ON ...` syntax.

LEFT JOIN

```sql
SELECT w.city, w.temp_lo, w.temp_hi, c.location
FROM weather w
LEFT JOIN cities c ON w.city = c.name;
```

- Missing matches on the right side produce `NULL` values.

Many-to-One Relationships

- Many rows in one table can point to one row in another table.
- PostgreSQL's foreign-key tutorial uses this pattern.

```
CREATE TABLE cities (
  name varchar(80) PRIMARY KEY
);
CREATE TABLE weather (
  city varchar(80) REFERENCES cities(name),
  date date
);
```

Many-to-Many Relationships

- PostgreSQL docs use a junction table for this pattern.
- One order can contain many products.
- One product can appear in many orders.

```
CREATE TABLE order_items (
  product_no integer REFERENCES products,
  order_id integer REFERENCES orders_docs,
  quantity integer,
  PRIMARY KEY (product_no, order_id)
);
```

Querying A Junction Table

```sql
SELECT oi.order_id, p.name, oi.quantity
FROM order_items oi
JOIN products p ON p.product_no = oi.product_no
ORDER BY oi.order_id, p.product_no;
```

Normalization

- Normalization reduces redundancy.
- It also reduces update, insert, and delete anomalies.
- Start with a clean normalized design.
- Denormalize only when a measured performance problem justifies it.

Primary Keys And Dependencies

- Start by asking: what uniquely identifies one row?
- In enrollments, the natural key is `(student_id, course_id)`.

```
student_id -> student_name
course_id -> course_name, lecturer_id
(student_id, course_id) -> grade
```

First Normal Form (1NF)

- 1NF means atomic values in each column.
- Avoid lists, repeated columns, or comma-separated values.

```
Bad:  student_id | student_name | courses
  1           | Alice         | Math, Physics
Good: enrollments(student_id, course_id)
```

- 1NF is a design rule, not a feature switch.

Second Normal Form (2NF)

- A table must already be in 1NF.
- Non-key columns must depend on the whole composite key.

```
Bad:  student_courses(student_id, student_name,
  course_id, course_name, lecturer_id, grade)
PK:   (student_id, course_id)
student_id -> student_name
course_id -> course_name, lecturer_id
```

- Good: split into students, courses, and enrollments.

Third Normal Form (3NF)

- 3NF removes transitive dependencies.

```
Bad:  courses(course_id, course_name, lecturer_id, lecturer_name)
course_id -> lecturer_id
lecturer_id -> lecturer_name
Good: courses(course_id, course_name, lecturer_id)
  lecturers(lecturer_id, lecturer_name)
```

Boyce-Codd Normal Form (BCNF)

- BCNF is stricter: every determinant must be a candidate key.

```
Bad:  teaching(student_id, course_id, lecturer_id)
(student_id, course_id) -> lecturer_id
lecturer_id -> course_id
lecturer_id is not a key, so BCNF fails.
Good: lecturers(lecturer_id, course_id)
  student_lecturers(student_id, lecturer_id)
```

How To Normalize Mathematically

1. List the attributes.
2. Write the functional dependencies.
3. Find the candidate key(s).
4. Remove repeating groups to reach 1NF.
5. Remove partial dependencies to reach 2NF.
6. Remove transitive dependencies to reach 3NF and BCNF.

Indexes

- An index is a data structure that speeds up lookups.
- A composite index can help with filtering and sorting together.

```
CREATE INDEX idx_orders_perf_cust_amount
ON orders_perf(cust_id, amount);

SELECT * FROM orders_perf
WHERE cust_id = 42
ORDER BY amount
LIMIT 5;
```

When To Add Indexes

- Good candidates: join keys, filter columns, foreign keys.
- Indexes can also support matching `ORDER BY` clauses.
- They are not free: extra storage and slower writes.
- Add them when a real query pattern needs them.

Summary

- Subqueries let one query depend on another.
- CTEs improve readability for multi-step SQL.
- Foreign keys model many-to-one relationships.
- Junction tables model many-to-many relationships.
- Primary keys and functional dependencies drive normalization.
- 1NF, 2NF, 3NF, and BCNF reduce different kinds of anomalies.
- Indexes improve performance for common access patterns.

EXERCISES

Exercise 1

- Use `books` and `authors` from `week2.sql`.
- Find books priced above the average price.
- Find books with titles longer than the average title length.
- Find books written by authors from the top author country.

Exercise 2

- Rewrite the Exercise 1 queries with CTEs.
- Compare readability with the subquery versions.
- Which version would you rather maintain later?

Exercise 3

- Use `LEFT JOIN` to list every author and a book count.
- Identify authors with no books.
- Explain why `COUNT(b.id)` returns `0` for those authors.

Exercise 4

- Start from `student_courses_denorm` in `week2.sql`.
- Create `students_norm`, `courses_norm`, and `enrollments_norm`.
- Migrate the data into the normalized tables.
- Explain whether your result reaches 2NF only or also 3NF.

Exercise 5

- Use `orders_perf` from `week2.sql`.
- Create an index on `cust_id`.
- Create a composite index on `(cust_id, amount)`.
- Run `EXPLAIN` on the filter and sort queries.

Reflection

1. What is the practical difference between a subquery and a CTE?
2. What is the difference between many-to-one and many-to-many?
3. What do `NULL` values in a `LEFT JOIN` usually tell you?
4. What is the difference between 3NF and BCNF?